JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Float

java.lang.Object
    java.lang.Number
        java.lang.Float

All Implemented Interfaces:

Serializable, Comparable<Float>, Constable, ConstantDesc

```
public final class Float
extends Number
implements Comparable<Float>, Constable, ConstantDesc
```

The `Float` class is the wrapper class for values of the primitive type `float`. An object of type `Float` contains a single field whose type is `float`.

In addition, this class provides several methods for converting a `float` to a `String` and a `String` to a `float`, as well as other constants and methods useful when dealing with a `float`.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Floating-point Equality, Equivalence, and Comparison

The class `java.lang.Double` has a discussion of equality, equivalence, and comparison of floating-point values that is equally applicable to `float` values.

Decimal ↔ Binary Conversion Issues

The discussion of binary to decimal conversion issues in `java.lang.Double` is also applicable to `float` values.

Since:

1.0

External Specifications

IEEE Standard for Floating-Point Arithmetic

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description

`static final int`	**`BYTES`**	The number of bytes used to represent a `float` value, 4.
`static final int`	**`MAX_EXPONENT`**	Maximum exponent a finite `float` variable may have, 127.
`static final float`	**`MAX_VALUE`**	A constant holding the largest positive finite value of type `float`, $(2-2^{-23})\cdot 2^{127}$.
`static final int`	**`MIN_EXPONENT`**	Minimum exponent a normalized `float` variable may have, -126.
`static final float`	**`MIN_NORMAL`**	A constant holding the smallest positive normal value of type `float`, 2^{-126}.
`static final float`	**`MIN_VALUE`**	A constant holding the smallest positive nonzero value of type `float`, 2^{-149}.
`static final float`	**`NaN`**	A constant holding a Not-a-Number (NaN) value of type `float`.
`static final float`	**`NEGATIVE_INFINITY`**	A constant holding the negative infinity of type `float`.
`static final float`	**`POSITIVE_INFINITY`**	A constant holding the positive infinity of type `float`.
`static final int`	**`PRECISION`**	The number of bits in the significand of a `float` value, 24.
`static final int`	**`SIZE`**	The number of bits used to represent a `float` value, 32.

static final Class<Float>	TYPE	The Class instance representing the primitive type float.

Constructor Summary

Constructors

Constructor	Description
Float(double value)	**Deprecated.** It is rarely appropriate to use this constructor.
Float(float value)	**Deprecated.** It is rarely appropriate to use this constructor.
Float(String s)	**Deprecated.** It is rarely appropriate to use this constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
byte	byteValue()	Returns the value of this Float as a byte after a narrowing primitive conversion.
static int	compare(float f1, float f2)	Compares the two specified float values.
int	compareTo (Float anotherFloat)	Compares two Float objects numerically.
Optional<Float>	describeConstable()	Returns an Optional containing the nominal descriptor for this instance, which is the instance itself.
double	doubleValue()	

		Returns the value of this `Float` as a `double` after a widening primitive conversion.
`boolean`	**`equals`**`(Object obj)`	Compares this object against the specified object.
`static float`	**`float16ToFloat`**`(short floatBinary16)`	Returns the `float` value closest to the numerical value of the argument, a floating-point binary16 value encoded in a `short`.
`static short`	**`floatToFloat16`**`(float f)`	Returns the floating-point binary16 value, encoded in a `short`, closest in value to the argument.
`static int`	**`floatToIntBits`**`(float value)`	Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "single format" bit layout.
`static int`	**`floatToRawIntBits`**`(float value)`	Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "single format" bit layout, preserving Not-a-Number (NaN) values.
`float`	**`floatValue`**`()`	Returns the `float` value of this `Float` object.
`int`	**`hashCode`**`()`	Returns a hash code for this `Float` object.
`static int`	**`hashCode`**`(float value)`	Returns a hash code for a `float` value;

		compatible with `Float.hashCode()`.
`static float`	**`intBitsToFloat`**`(int bits)`	Returns the `float` value corresponding to a given bit representation.
`int`	**`intValue`**`()`	Returns the value of this `Float` as an `int` after a narrowing primitive conversion.
`static boolean`	**`isFinite`**`(float f)`	Returns `true` if the argument is a finite floating-point value; returns `false` otherwise (for NaN and infinity arguments).
`boolean`	**`isInfinite`**`()`	Returns `true` if this `Float` value is infinitely large in magnitude, `false` otherwise.
`static boolean`	**`isInfinite`**`(float v)`	Returns `true` if the specified number is infinitely large in magnitude, `false` otherwise.
`boolean`	**`isNaN`**`()`	Returns `true` if this `Float` value is a Not-a-Number (NaN), `false` otherwise.
`static boolean`	**`isNaN`**`(float v)`	Returns `true` if the specified number is a Not-a-Number (NaN) value, `false` otherwise.
`long`	**`longValue`**`()`	Returns value of this `Float` as a `long` after a narrowing primitive conversion.

static float	**max**(float a, float b)	Returns the greater of two float values as if by calling Math.max.
static float	**min**(float a, float b)	Returns the smaller of two float values as if by calling Math.min.
static float	**parseFloat**(String s)	Returns a new float initialized to the value represented by the specified String, as performed by the valueOf method of class Float.
Float	**resolveConstantDesc**(MethodHandles.Lookup lo	Resolves this instance as a ConstantDesc, the result of which is the instance itself.
short	**shortValue**()	Returns the value of this Float as a short after a narrowing primitive conversion.
static float	**sum**(float a, float b)	Adds two float values together as per the + operator.
static String	**toHexString**(float f)	Returns a hexadecimal string representation of the float argument.
String	**toString**()	Returns a string representation of this Float object.
static String	**toString**(float f)	Returns a string representation of the float argument.
static Float	**valueOf**(float f)	Returns a Float instance representing the specified float value.
static Float	**valueOf**(String s)	

	Returns a `Float` object holding the `float` value represented by the argument string `s`.

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

POSITIVE_INFINITY

```
public static final float POSITIVE_INFINITY
```

A constant holding the positive infinity of type `float`. It is equal to the value returned by `Float.intBitsToFloat(0x7f800000)`.

See Also:

Constant Field Values

NEGATIVE_INFINITY

```
public static final float NEGATIVE_INFINITY
```

A constant holding the negative infinity of type `float`. It is equal to the value returned by `Float.intBitsToFloat(0xff800000)`.

See Also:

Constant Field Values

NaN

```
public static final float NaN
```

A constant holding a Not-a-Number (NaN) value of type `float`. It is equivalent to the value returned by`Float.intBitsToFloat(0x7fc00000)`.

See Also:

Constant Field Values

MAX_VALUE

```
public static final float MAX_VALUE
```

A constant holding the largest positive finite value of type `float`, $(2-2^{-23})\cdot 2^{127}$. It is equal to the hexadecimal floating-point literal `0x1.fffffeP+127f` and also equal to `Float.intBitsToFloat(0x7f7fffff)`.

See Also:

Constant Field Values

MIN_NORMAL

```
public static final float MIN_NORMAL
```

A constant holding the smallest positive normal value of type `float`, 2^{-126}. It is equal to the hexadecimal floating-point literal `0x1.0p-126f` and also equal to `Float.intBitsToFloat(0x00800000)`.

Since:

1.6

See Also:

Constant Field Values

MIN_VALUE

```
public static final float MIN_VALUE
```

A constant holding the smallest positive nonzero value of type `float`, 2^{-149}. It is equal to the hexadecimal floating-point literal `0x0.000002P-126f` and also equal to `Float.intBitsToFloat(0x1)`.

See Also:

Constant Field Values

SIZE

```
public static final int SIZE
```

The number of bits used to represent a `float` value, 32.

Since:

1.5

See Also:

Constant Field Values

PRECISION

```
public static final int PRECISION
```

The number of bits in the significand of a `float` value, 24. This is the parameter N in section §4.2.3 of *The Java Language Specification*.

Since:

19

See Also:

Constant Field Values

MAX_EXPONENT

```
public static final int MAX_EXPONENT
```

Maximum exponent a finite `float` variable may have, 127. It is equal to the value returned by `Math.getExponent(Float.MAX_VALUE)`.

Since:

1.6

See Also:

Constant Field Values

MIN_EXPONENT

```
public static final int MIN_EXPONENT
```

Minimum exponent a normalized `float` variable may have, -126. It is equal to the value returned by `Math.getExponent(Float.MIN_NORMAL)`.

Since:

1.6

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `float` value, 4.

Since:

1.8

See Also:

Constant Field Values

TYPE

```
public static final Class<Float> TYPE
```

The `Class` instance representing the primitive type `float`.

Since:

1.1

Constructor Details

Float

```
@Deprecated(since="9")
public Float(float value)
```

Deprecated.
It is rarely appropriate to use this constructor. The static factory `valueOf(float)` is generally a better choice, as it is likely to yield significantly better space and time performance.

Constructs a newly allocated `Float` object that represents the primitive `float` argument.

Parameters:

`value` - the value to be represented by the `Float`.

Float

```
@Deprecated(since="9")
public Float(double value)
```

Deprecated.
It is rarely appropriate to use this constructor. Instead, use the static factory method `valueOf(float)` method as follows: `Float.valueOf((float)value)`.

Constructs a newly allocated `Float` object that represents the argument converted to type `float`.

Parameters:

`value` - the value to be represented by the `Float`.

Float

```
@Deprecated(since="9")
public Float(String s)
      throws NumberFormatException
```

Deprecated.
It is rarely appropriate to use this constructor. Use `parseFloat(String)` to convert a string to a `float` primitive, or use `valueOf(String)` to convert a string to a `Float` object.

Constructs a newly allocated `Float` object that represents the floating-point value of type `float` represented by the string. The string is converted to a `float` value as if by the `valueOf` method.

Parameters:

`s` - a string to be converted to a `Float`.

Throws:

`NumberFormatException` - if the string does not contain a parsable number.

Method Details

toString

`public static String toString(float f)`

Returns a string representation of the `float` argument. All characters mentioned below are ASCII characters.

- If the argument is NaN, the result is the string `"NaN"`.
- Otherwise, the result is a string that represents the sign and magnitude (absolute value) of the argument. If the sign is negative, the first character of the result is `'-'` (`'\u002D'`); if the sign is positive, no sign character appears in the result. As for the magnitude *m*:
  - If *m* is infinity, it is represented by the characters `"Infinity"`; thus, positive infinity produces the result `"Infinity"` and negative infinity produces the result `"-Infinity"`.
  - If *m* is zero, it is represented by the characters `"0.0"`; thus, negative zero produces the result `"-0.0"` and positive zero produces the result `"0.0"`.
  - Otherwise *m* is positive and finite. It is converted to a string in two stages:
    - *Selection of a decimal*: A well-defined decimal d_m is selected to represent *m*. This decimal is (almost always) the *shortest* one that rounds to *m* according to the round to nearest rounding policy of IEEE 754 floating-point arithmetic.
    - *Formatting as a string*: The decimal d_m is formatted as a string, either in plain or in computerized scientific notation, depending on its value.

A *decimal* is a number of the form $s \times 10^i$ for some (unique) integers $s > 0$ and i such that s is not a multiple of 10. These integers are the *significand* and the *exponent*, respectively, of the decimal. The *length* of the decimal is the (unique) positive integer n meeting $10^{n-1} \le s < 10^n$.

The decimal d_m for a finite positive m is defined as follows:

- Let R be the set of all decimals that round to m according to the usual *round to nearest* rounding policy of IEEE 754 floating-point arithmetic.
- Let p be the minimal length over all decimals in R.
- When $p \ge 2$, let T be the set of all decimals in R with length p. Otherwise, let T be the set of all decimals in R with length 1 or 2.
- Define d_m as the decimal in T that is closest to m. Or if there are two such decimals in T, select the one with the even significand.

The (uniquely) selected decimal d_m is then formatted. Let s, i and n be the significand, exponent and length of d_m, respectively. Further, let $e = n + i - 1$ and let $s_1 \ldots s_n$ be the usual decimal expansion of s. Note that $s_1 \neq 0$ and $s_n \neq 0$. Below, the decimal point `'.'` is `'\u002E'` and the exponent indicator `'E'` is `'\u0045'`.

- Case $-3 \le e < 0$: d_m is formatted as `0.0`$\ldots$`0`$s_1 \ldots s_n$, where there are exactly $-(n + i)$ zeroes between the decimal point and s_1. For example, 123×10^{-4} is formatted as `0.0123`.
- Case $0 \le e < 7$:
  - Subcase $i \ge 0$: d_m is formatted as $s_1 \ldots s_n$`0`$\ldots$`0.0`, where there are exactly i zeroes between s_n and the decimal point. For example, 123×10^2 is formatted as `12300.0`.
  - Subcase $i < 0$: d_m is formatted as $s_1 \ldots s_{n+i}$`.`$s_{n+i+1} \ldots s_n$, where there are exactly $-i$ digits to the right of the decimal point. For example, 123×10^{-1} is formatted as `12.3`.
- Case $e < -3$ or $e \ge 7$: computerized scientific notation is used to format d_m. Here e is formatted as by `Integer.toString(int)`.
  - Subcase $n = 1$: d_m is formatted as s_1`.0E`e. For example, 1×10^{23} is formatted as `1.0E23`.
  - Subcase $n > 1$: d_m is formatted as s_1`.`$s_2 \ldots s_n$`E`e. For example, 123×10^{-21} is formatted as `1.23E-19`.

To create localized string representations of a floating-point value, use subclasses of `NumberFormat`.

API Note:

This method corresponds to the general functionality of the convertToDecimalCharacter operation defined in IEEE 754; however, that operation is defined in terms of specifying the number of significand digits used in the conversion. Code to do such a conversion in the Java platform includes converting the `float` to a `BigDecimal` exactly and then rounding the `BigDecimal` to the desired number of digits; sample code:

```
floatf = 0.1f;
int digits = 15;
BigDecimal bd = new BigDecimal(f);
String result = bd.round(new MathContext(digits,  RoundingMode.HALF_UP));
// 0.100000001490116
```

Parameters:

`f` - the `float` to be converted.

Returns:

a string representation of the argument.

toHexString

```
public static String toHexString(float f)
```

Returns a hexadecimal string representation of the `float` argument. All characters mentioned below are ASCII characters.

- If the argument is NaN, the result is the string `"NaN"`.
- Otherwise, the result is a string that represents the sign and magnitude (absolute value) of the argument. If the sign is negative, the first character of the result is `'-'` (`'\u002D'`); if the sign is positive, no sign character appears in the result. As for the magnitude *m*:
    - If *m* is infinity, it is represented by the string `"Infinity"`; thus, positive infinity produces the result `"Infinity"` and negative infinity produces the result `"-Infinity"`.
    - If *m* is zero, it is represented by the string `"0x0.0p0"`; thus, negative zero produces the result `"-0x0.0p0"` and positive zero produces the result `"0x0.0p0"`.
    - If *m* is a `float` value with a normalized representation, substrings are used to represent the significand and exponent fields. The significand is represented by the characters `"0x1."` followed by a lowercase hexadecimal representation of the rest of the significand as a fraction. Trailing zeros in the hexadecimal representation are removed unless all the digits are zero, in which case a single zero is used. Next, the exponent is represented by `"p"` followed by a

decimal string of the unbiased exponent as if produced by a call to `Integer.toString` on the exponent value.

- If *m* is a `float` value with a subnormal representation, the significand is represented by the characters `"0x0."` followed by a hexadecimal representation of the rest of the significand as a fraction. Trailing zeros in the hexadecimal representation are removed. Next, the exponent is represented by `"p-126"`. Note that there
must be at least one nonzero digit in a subnormal significand.

Examples

Floating-point Value	Hexadecimal String
`1.0`	`0x1.0p0`
`-1.0`	`-0x1.0p0`
`2.0`	`0x1.0p1`
`3.0`	`0x1.8p1`
`0.5`	`0x1.0p-1`
`0.25`	`0x1.0p-2`
`Float.MAX_VALUE`	`0x1.fffffep127`
`Minimum Normal Value`	`0x1.0p-126`
`Maximum Subnormal Value`	`0x0.fffffep-126`
`Float.MIN_VALUE`	`0x0.000002p-126`

API Note:

This method corresponds to the convertToHexCharacter operation defined in IEEE 754.

Parameters:

`f` - the `float` to be converted.

Returns:

a hex string representation of the argument.

Since:

1.5

valueOf

```
public static Float valueOf(String s)
                     throws NumberFormatException
```

Returns a `Float` object holding the `float` value represented by the argument string `s`.

If `s` is `null`, then a `NullPointerException` is thrown.

Leading and trailing whitespace characters in `s` are ignored. Whitespace is removed as if by the `String.trim()` method; that is, both ASCII space and control characters are removed. The rest of `s` should constitute a *FloatValue* as described by the lexical syntax rules:

FloatValue:
- $Sign_{opt}$ `NaN`
- $Sign_{opt}$ `Infinity`
- $Sign_{opt}$ *FloatingPointLiteral*
- $Sign_{opt}$ *HexFloatingPointLiteral*
- *SignedInteger*

HexFloatingPointLiteral:
- *HexSignificand BinaryExponent* $FloatTypeSuffix_{opt}$

HexSignificand:
- *HexNumeral*
- *HexNumeral* `.`
- `0x` $HexDigits_{opt}$ `.` *HexDigits*
- `0X` $HexDigits_{opt}$ `.` *HexDigits*

BinaryExponent:
- *BinaryExponentIndicator SignedInteger*

BinaryExponentIndicator:
- `p`
- `P`

where *Sign*, *FloatingPointLiteral*, *HexNumeral*, *HexDigits*, *SignedInteger* and *FloatTypeSuffix* are as defined in the lexical structure sections of *The Java Language Specification*, except that underscores are not accepted between digits. If `s` does not have the form of a *FloatValue*, then a `NumberFormatException` is thrown. Otherwise, `s` is regarded as representing an exact decimal value in the usual "computerized scientific notation" or as an exact hexadecimal value; this exact numerical value is then conceptually converted to an "infinitely precise" binary value that is then rounded to type `float` by the usual round-to-nearest rule of IEEE 754 floating-point arithmetic, which includes preserving the sign of a zero value. Note that the round-to-nearest rule also implies overflow and underflow behaviour; if the exact value of `s` is large enough in magnitude (greater than or equal to (`MAX_VALUE` + `ulp(MAX_VALUE)`/2), rounding to `float` will result in an infinity and if the exact value of `s` is small enough in magnitude (less than or equal to `MIN_VALUE`/2), rounding to float will result in a zero. Finally, after rounding a `Float` object representing this `float` value is returned.

Note that trailing format specifiers, specifiers that determine the type of a floating-point literal (`1.0f` is a `float` value; `1.0d` is a `double` value), do *not* influence the results of this method. In other words, the numerical value of

the input string is converted directly to the target floating-point type. In general, the two-step sequence of conversions, string to `double` followed by `double` to `float`, is *not* equivalent to converting a string directly to `float`. For example, if first converted to an intermediate `double` and then to `float`, the string
`"1.00000017881393421514957253748434595763683319091796875001d"`
results in the `float` value `1.0000002f`; if the string is converted directly to `float`, `1.0000001f` results.

To avoid calling this method on an invalid string and having a `NumberFormatException` be thrown, the documentation for `Double.valueOf` lists a regular expression which can be used to screen the input.

API Note:

To interpret localized string representations of a floating-point value, or string representations that have non-ASCII digits, use `NumberFormat`. For example,

```
NumberFormat.getInstance(l).parse(s).floatValue();
```

where `l` is the desired locale, or `Locale.ROOT` if locale insensitive., This method corresponds to the convertFromDecimalCharacter and convertFromHexCharacter operations defined in IEEE 754.

Parameters:

`s` - the string to be parsed.

Returns:

a `Float` object holding the value represented by the `String` argument.

Throws:

`NumberFormatException` - if the string does not contain a parsable number.

See Also:

Decimal ↔ Binary Conversion Issues

valueOf

```
public static Float valueOf(float f)
```

Returns a `Float` instance representing the specified `float` value. If a new `Float` instance is not required, this method should generally be used in preference to the constructor `Float(float)`, as this method is likely to yield significantly better space and time performance by caching frequently requested values.

Parameters:

`f` - a float value.

Returns:

a `Float` instance representing `f`.

Since:

1.5

parseFloat

```
public static float parseFloat(String s)
                        throws NumberFormatException
```

Returns a new `float` initialized to the value represented by the specified `String`, as performed by the `valueOf` method of class `Float`.

Parameters:

`s` - the string to be parsed.

Returns:

the `float` value represented by the string argument.

Throws:

`NullPointerException` - if the string is null

`NumberFormatException` - if the string does not contain a parsable `float`.

Since:

1.2

See Also:

`valueOf(String)`,
Decimal ↔ Binary Conversion Issues

isNaN

```
public static boolean isNaN(float v)
```

Returns `true` if the specified number is a Not-a-Number (NaN) value, `false` otherwise.

API Note:

This method corresponds to the isNaN operation defined in IEEE 754.

Parameters:

`v` - the value to be tested.

Returns:

`true` if the argument is NaN; `false` otherwise.

isInfinite

```
public static boolean isInfinite(float v)
```

Returns `true` if the specified number is infinitely large in magnitude, `false` otherwise.

API Note:

This method corresponds to the isInfinite operation defined in IEEE 754.

Parameters:

`v` - the value to be tested.

Returns:

`true` if the argument is positive infinity or negative infinity; `false` otherwise.

isFinite

```
public static boolean isFinite(float f)
```

Returns `true` if the argument is a finite floating-point value; returns `false` otherwise (for NaN and infinity arguments).

API Note:

This method corresponds to the isFinite operation defined in IEEE 754.

Parameters:

`f` - the `float` value to be tested

Returns:

`true` if the argument is a finite floating-point value, `false` otherwise.

Since:

1.8

isNaN

```
public boolean isNaN()
```

Returns `true` if this `Float` value is a Not-a-Number (NaN), `false` otherwise.

Returns:

`true` if the value represented by this object is NaN; `false` otherwise.

isInfinite

```
public boolean isInfinite()
```

Returns `true` if this `Float` value is infinitely large in magnitude, `false` otherwise.

Returns:

`true` if the value represented by this object is positive infinity or negative infinity; `false` otherwise.

toString

```
public String toString()
```

Returns a string representation of this `Float` object. The primitive `float` value represented by this object is converted to a `String` exactly as if by the method `toString` of one argument.

Overrides:

`toString` in class `Object`

Returns:

a `String` representation of this object.

See Also:

`toString(float)`

byteValue

```
public byte byteValue()
```

Returns the value of this `Float` as a `byte` after a narrowing primitive conversion.

Overrides:

`byteValue` in class `Number`

Returns:

the `float` value represented by this object converted to type `byte`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

shortValue

```
public short shortValue()
```

Returns the value of this `Float` as a `short` after a narrowing primitive conversion.

Overrides:

`shortValue` in class `Number`

Returns:

the `float` value represented by this object converted to type `short`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

Since:

1.1

intValue

```
public int intValue()
```

Returns the value of this `Float` as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

API Note:

This method corresponds to the convertToIntegerTowardZero operation defined in IEEE 754.

Returns:

the `float` value represented by this object converted to type `int`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

longValue

```
public long longValue()
```

Returns value of this `Float` as a `long` after a narrowing primitive conversion.

Specified by:

`longValue` in class `Number`

API Note:

This method corresponds to the convertToIntegerTowardZero operation defined in IEEE 754.

Returns:

the `float` value represented by this object converted to type `long`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

floatValue

```
public float floatValue()
```

Returns the `float` value of this `Float` object.

Specified by:

`floatValue` in class `Number`

Returns:

the `float` value represented by this object

doubleValue

```
public double doubleValue()
```

Returns the value of this `Float` as a `double` after a widening primitive conversion.

Specified by:

`doubleValue` in class `Number`

API Note:

This method corresponds to the convertFormat operation defined in IEEE 754.

Returns:

the `float` value represented by this object converted to type `double`

See *Java Language Specification*:

5.1.2 Widening Primitive Conversion

hashCode

```
public int hashCode()
```

Returns a hash code for this `Float` object. The result is the integer bit representation, exactly as produced by the method `floatToIntBits(float)`, of the primitive `float` value represented by this `Float` object.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

hashCode

```
public static int hashCode(float value)
```

Returns a hash code for a `float` value; compatible with `Float.hashCode()`.

Parameters:

`value` - the value to hash

Returns:

a hash code value for a `float` value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object against the specified object. The result is `true` if and only if the argument is not `null` and is a `Float` object that represents a `float` with the same value as the `float` represented by this object. For this purpose, two `float` values are considered to be the same if and only if the method `floatToIntBits(float)` returns the identical `int` value when applied to each. In other words, representation equivalence is used to compare the `float` values.

Overrides:

`equals` in class `Object`

API Note:

This method is defined in terms of `floatToIntBits(float)` rather than the == operator on `float` values since the == operator does *not* define an equivalence relation and to satisfy the equals contract an equivalence relation must be implemented; see this discussion for details of floating-point equality and equivalence.

Parameters:

`obj` - the object to be compared

Returns:

`true` if the objects are the same; `false` otherwise.

See *Java Language Specification*:

15.21.1 Numerical Equality Operators == and !=

See Also:

`floatToIntBits(float)`

floatToIntBits

```
public static int floatToIntBits(float value)
```

Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "single format" bit layout.

Bit 31 (the bit that is selected by the mask `0x80000000`) represents the sign of the floating-point number. Bits 30-23 (the bits that are selected by the mask `0x7f800000`) represent the exponent. Bits 22-0 (the bits that are selected by the mask `0x007fffff`) represent the significand (sometimes called the mantissa) of the floating-point number.

If the argument is positive infinity, the result is `0x7f800000`.

If the argument is negative infinity, the result is `0xff800000`.

If the argument is NaN, the result is `0x7fc00000`.

In all cases, the result is an integer that, when given to the `intBitsToFloat(int)` method, will produce a floating-point value the same as the argument to `floatToIntBits` (except all NaN values are collapsed to a single "canonical" NaN value).

Parameters:

`value` - a floating-point number.

Returns:

the bits that represent the floating-point number.

floatToRawIntBits

```
public static int floatToRawIntBits(float value)
```

Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "single format" bit layout, preserving Not-a-Number (NaN) values.

Bit 31 (the bit that is selected by the mask `0x80000000`) represents the sign of the floating-point number. Bits 30-23 (the bits that are selected by the mask `0x7f800000`) represent the exponent. Bits 22-0 (the bits that are selected by the mask `0x007fffff`) represent the significand (sometimes called the mantissa) of the floating-point number.

If the argument is positive infinity, the result is `0x7f800000`.

If the argument is negative infinity, the result is `0xff800000`.

If the argument is NaN, the result is the integer representing the actual NaN value. Unlike the `floatToIntBits` method, `floatToRawIntBits` does not collapse all the bit patterns encoding a NaN to a single "canonical" NaN value.

In all cases, the result is an integer that, when given to the `intBitsToFloat(int)` method, will produce a floating-point value the same as the argument to `floatToRawIntBits`.

Parameters:

`value` - a floating-point number.

Returns:

the bits that represent the floating-point number.

Since:

1.3

intBitsToFloat

```
public static float intBitsToFloat(int bits)
```

Returns the `float` value corresponding to a given bit representation. The argument is considered to be a representation of a floating-point value according to the IEEE 754 floating-point "single format" bit layout.

If the argument is `0x7f800000`, the result is positive infinity.

If the argument is `0xff800000`, the result is negative infinity.

If the argument is any value in the range `0x7f800001` through `0x7fffffff` or in the range `0xff800001` through `0xffffffff`, the result is a NaN. No IEEE 754 floating-point operation provided by Java can distinguish between two NaN values of the same type with different bit patterns. Distinct values of NaN are only distinguishable by use of the `Float.floatToRawIntBits` method.

In all other cases, let *s*, *e*, and *m* be three values that can be computed from the argument:

```
int s = ((bits >> 31) == 0) ? 1 : -1;
int e = ((bits >> 23) & 0xff);
int m = (e == 0) ?
                (bits & 0x7fffff) << 1 :
                (bits & 0x7fffff) | 0x800000;
```

Then
the floating-point result equals the value of the mathematical expression $s \cdot m \cdot 2^{e-150}$.

Note that this method may not be able to return a `float` NaN with exactly same bit pattern as the `int` argument. IEEE 754 distinguishes between two kinds of NaNs, quiet NaNs and *signaling NaNs*. The differences between the two kinds of NaN are generally not visible in Java. Arithmetic operations on signaling NaNs turn them into quiet NaNs with a different, but often similar, bit pattern. However, on some processors merely copying a signaling NaN also performs that conversion. In particular, copying a signaling NaN to return it to the calling method may perform this conversion. So `intBitsToFloat` may not be able to return a `float` with a signaling NaN bit pattern. Consequently, for some `int` values, `floatToRawIntBits(intBitsToFloat(start))` may *not* equal `start`. Moreover, which particular bit patterns represent signaling NaNs is platform dependent;

although all NaN bit patterns, quiet or signaling, must be in the NaN range identified above.

Parameters:

`bits` - an integer.

Returns:

the `float` floating-point value with the same bit pattern.

float16ToFloat

```
public static float float16ToFloat(short floatBinary16)
```

Returns the `float` value closest to the numerical value of the argument, a floating-point binary16 value encoded in a `short`. The conversion is exact; all binary16 values can be exactly represented in `float`. Special cases:

- If the argument is zero, the result is a zero with the same sign as the argument.
- If the argument is infinite, the result is an infinity with the same sign as the argument.
- If the argument is a NaN, the result is a NaN.

IEEE 754 binary16 format

The IEEE 754 standard defines binary16 as a 16-bit format, along with the 32-bit binary32 format (corresponding to the `float` type) and the 64-bit binary64 format (corresponding to the `double` type). The binary16 format is similar to the other IEEE 754 formats, except smaller, having all the usual IEEE 754 values such as NaN, signed infinities, signed zeros, and subnormals. The parameters (JLS §4.2.3) for the binary16 format are N = 11 precision bits, K = 5 exponent bits, E_{max} = 15, and E_{min} = -14.

API Note:

This method corresponds to the convertFormat operation defined in IEEE 754 from the binary16 format to the binary32 format. The operation of this method is analogous to a primitive widening conversion (JLS §5.1.2).

Parameters:

`floatBinary16` - the binary16 value to convert to `float`

Returns:

the `float` value closest to the numerical value of the argument, a floating-point binary16 value encoded in a `short`

Since:

20

floatToFloat16

```
public static short floatToFloat16(float f)
```

Returns the floating-point binary16 value, encoded in a `short`, closest in value to the argument. The conversion is computed under the round to nearest even rounding mode. Special cases:

- If the argument is zero, the result is a zero with the same sign as the argument.
- If the argument is infinite, the result is an infinity with the same sign as the argument.
- If the argument is a NaN, the result is a NaN.

The binary16 format is discussed in more detail in the `float16ToFloat(short)` method.

API Note:

This method corresponds to the convertFormat operation defined in IEEE 754 from the binary32 format to the binary16 format. The operation of this method is analogous to a primitive narrowing conversion (JLS §5.1.3).

Parameters:

`f` - the `float` value to convert to binary16

Returns:

the floating-point binary16 value, encoded in a `short`, closest in value to the argument

Since:

20

compareTo

```
public int compareTo(Float anotherFloat)
```

Compares two `Float` objects numerically. This method imposes a total order on `Float` objects with two differences compared to the incomplete order defined by the Java language numerical comparison operators (`<, <=, ==, >=, >`) on `float` values.

- A NaN is *unordered* with respect to other values and unequal to itself under the comparison operators. This method chooses to define `Float.NaN` to be equal to itself and greater than all other `double` values (including `Float.POSITIVE_INFINITY`).
- Positive zero and negative zero compare equal numerically, but are distinct and distinguishable values. This method chooses to define positive zero (`+0.0f`), to be greater than negative zero (`-0.0f`).

This ensures that the *natural ordering* of `Float` objects imposed by this method is *consistent with equals*; see this discussion for details of floating-point comparison and ordering.

Specified by:

`compareTo` in interface `Comparable<Float>`

API Note:

For a discussion of differences between the total order of this method compared to the total order defined by the IEEE 754 standard, see the note in `Double.compareTo(Double)`.

Parameters:

`anotherFloat` - the `Float` to be compared.

Returns:

the value `0` if `anotherFloat` is numerically equal to this `Float`; a value less than `0` if this `Float` is numerically less than `anotherFloat`; and a value greater than `0` if this `Float` is numerically greater than `anotherFloat`.

See *Java Language Specification*:

15.20.1 Numerical Comparison Operators <, <=, >, and >=

Since:

1.2

compare

```
public static int compare(float f1,
                          float f2)
```

Compares the two specified `float` values. The sign of the integer value returned is the same as that of the integer that would be returned by the call:

```
Float.valueOf(f1).compareTo(Float.valueOf(f2))
```

API Note:

One idiom to implement representation equivalence on `float` values is

```
Float.compare(a, b) == 0
```

Parameters:

`f1` - the first `float` to compare.

`f2` - the second `float` to compare.

Returns:

the value `0` if `f1` is numerically equal to `f2`; a value less than `0` if `f1` is numerically less than `f2`; and a value greater than `0` if `f1` is numerically greater than `f2`.

Since:

1.4

sum

```
public static float sum(float a,
                        float b)
```

Adds two `float` values together as per the + operator.

API Note:

This method corresponds to the addition operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the sum of `a` and `b`

See *Java Language Specification*:

4.2.4 Floating-Point Operations

Since:

1.8

See Also:

`BinaryOperator`

max

```
public static float max(float a,
                        float b)
```

Returns the greater of two `float` values as if by calling `Math.max`.

API Note:

This method corresponds to the maximum operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the greater of `a` and `b`

Since:

1.8

See Also:

BinaryOperator

min

```
public static float min(float a,
                        float b)
```

Returns the smaller of two `float` values as if by calling `Math.min`.

API Note:

This method corresponds to the minimum operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the smaller of `a` and `b`

Since:

1.8

See Also:

BinaryOperator

describeConstable

```
public Optional<Float> describeConstable()
```

Returns an `Optional` containing the nominal descriptor for this instance, which is the instance itself.

Specified by:

`describeConstable` in interface `Constable`

Returns:

an `Optional` describing the Float instance

Since:

12

resolveConstantDesc

```
public Float resolveConstantDesc(MethodHandles.Lookup lookup)
```

Resolves this instance as a `ConstantDesc`, the result of which is the instance itself.

Specified by:

`resolveConstantDesc` in interface `ConstantDesc`

Parameters:

`lookup` - ignored

Returns:

the `Float` instance

Since:

12

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.